

Docker & Docker Compose

Comprendre les conteneurs, installer Docker, lire notre `docker-compose.yml` ligne par ligne, brancher phpMyAdmin dans le navigateur — et ranger WAMP au placard.



Au programme

1. Docker, c'est quoi ? (le CM)
2. Installer Docker (Linux / Mac / Windows)
3. Docker Desktop
4. Le Dockerfile
5. Partir direct du YAML : `image` ou `build` ?
6. Notre `docker-compose.yml`
7. Le YAML décortiqué
8. Les ports, pour de bon
9. phpMyAdmin dans le navigateur
10. Adieu WAMP / XAMPP
11. Mini-TP : TP0 + TP1 avec Docker
12. Récap des commandes

01 Docker, c'est quoi ?

Le problème qu'on cherche à régler

Une application, ce n'est jamais « juste du code ». C'est du code *plus* une version précise de PHP, *plus* Apache configuré d'une certaine façon, *plus* une base MySQL, *plus* des extensions installées. Sur votre machine, tout marche. Sur celle du voisin, PHP est en 7.4 et rien ne fonctionne. Sur le serveur, l'extension manque. C'est le fameux :

⚠ « Ça marche sur MA machine »

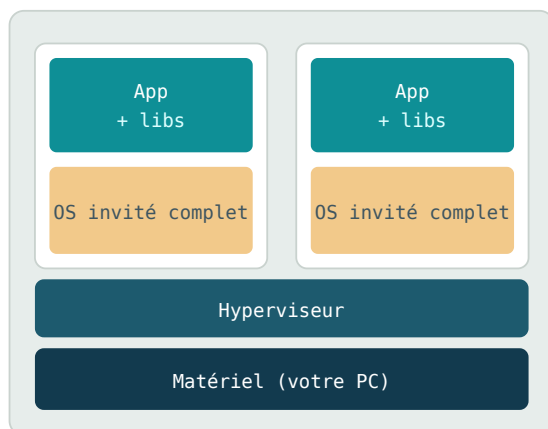
La phrase que tout développeur a dite au moins une fois. Le problème n'est pas le code : c'est **l'environnement autour du code** qui diffère d'un poste à l'autre.

Docker répond à ça simplement : on emballe l'application **et tout son environnement** dans un colis standardisé qui se comporte exactement pareil partout. Ce colis, c'est le **conteneur**.

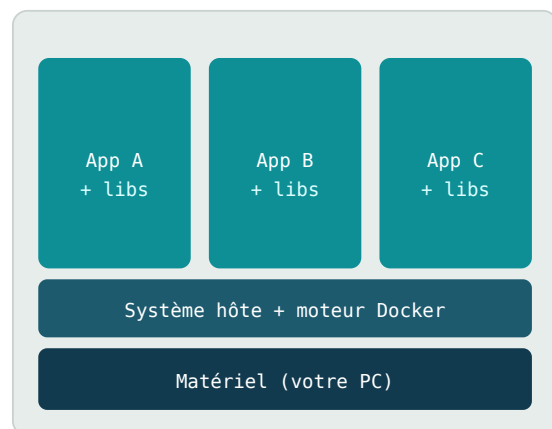
Machine virtuelle ou conteneur ?

On confond souvent les deux. La différence tient en une image : la VM emporte tout un système d'exploitation, le conteneur non.

Machine virtuelle



Conteneurs Docker



À gauche : chaque VM traîne son propre OS (lourd, lent). À droite : les conteneurs partagent le noyau de l'hôte (légers, démarrent en secondes).

■ Un conteneur

Un processus isolé qui embarque son application et ses dépendances, mais **partage le noyau** du système hôte. Résultat : quelques Mo au lieu de plusieurs Go, un démarrage en une ou deux secondes, un comportement identique partout.

Le vocabulaire à ne jamais confondre

Notion	C'est quoi	Analogie
Image	Un modèle figé, en lecture seule (ex. <code>php:8.2-apache</code>). On la télécharge, on ne la modifie pas.	Le disque d'installation
Conteneur	Une instance <i>qui tourne</i> , créée à partir d'une image. Votre « mini-PC allumé ».	Le PC une fois installé et démarré
Volume	Un dossier partagé entre votre machine et le conteneur. Fait vivre le code ou les données au-delà du conteneur.	Une clé USB branchée sur le mini-PC
Registre	La bibliothèque en ligne d'où l'on télécharge les images. Le principal est <i>Docker Hub</i> .	Le magasin d'applications
Service	Un conteneur décrit dans le <code>docker-compose.yml</code> (ex. <code>web</code> , <code>db</code> , <code>phpmyadmin</code>).	Le nom donné à la machine

📌 À retenir

Image → modèle. Conteneur → image lancée. Une image peut lancer autant de conteneurs qu'on veut, comme un même disque d'installation sert à installer dix PC identiques.

02 Installer Docker sur son poste

Le but : avoir la commande `docker` fonctionnelle dans un terminal. La vérification est la même partout, on la fait à la fin :

```
docker --version
docker compose version
docker run --rm hello-world # doit afficher "Hello from Docker!"
```



Linux

Ubuntu / Debian

Sur Linux, Docker tourne nativement : pas besoin de Docker Desktop. On installe le moteur (*Docker Engine*).

```
# Script officiel, le plus simple pour un poste de dev
curl -fsSL https://get.docker.com | sudo sh
```

```
# Pour lancer docker sans sudo (se reconnecter après) :
sudo usermod -aG docker $USER
```

💡 Astuce

Après le `usermod`, **déconnectez-vous puis reconnectez-vous** (ou `newgrp docker`). Sinon « permission denied » sur la socket Docker.

macOS Intel & Apple Silicon

On installe **Docker Desktop pour Mac**, qui embarque une petite VM Linux (le moteur Docker a besoin de Linux).

1. Télécharger Docker Desktop sur docker.com en choisissant la bonne puce : **Apple Silicon** (M1/M2/M3...) ou **Intel**.
2. Glisser Docker dans *Applications*, puis le lancer une première fois.
3. Attendre l'icône 🐳 « verte » dans la barre de menus.

Windows WSL2

Sur Windows, Docker passe par **WSL2** (un vrai noyau Linux intégré à Windows). Docker Desktop s'appuie dessus.

```
# Dans un PowerShell "en administrateur", une seule fois :
wsl --install
# Redémarrer, puis installer Docker Desktop pour Windows.
```

📌 Bon à savoir

Dans Docker Desktop → *Settings* → *Resources* → *WSL integration*, activez votre distribution (Ubuntu). Vous pourrez alors taper `docker` directement depuis le terminal Ubuntu WSL, bien plus agréable que PowerShell.

⚠ Le piège n°1 de l'installation

Sur Mac et Windows, **Docker ne marche que si Docker Desktop est lancé**. Si `docker ps` répond « Cannot connect to the Docker daemon », ce n'est pas cassé : l'application n'est simplement pas démarrée.

03 Docker Desktop

Docker Desktop est l'application « tout-en-un » sur Mac et Windows. Elle fait trois choses :

- elle fait tourner le **moteur Docker** (le *daemon*) dans une VM Linux invisible ;
- elle fournit la **commande** `docker` dans votre terminal ;
- elle ajoute une **interface graphique** pour voir ce qui tourne sans taper de commandes.

🎯 Ce que vous verrez dans l'interface

Containers : vos conteneurs et leur statut (vert = *running*), un bouton pour ouvrir un terminal dedans, voir les logs, arrêter. **Images** : les images téléchargées et leur poids. **Volumes** : les données persistées.

Tout ce que fait l'interface, vous pouvez le faire en ligne de commande — et inversement. Les deux pilotent le **même** moteur. En cours on privilégie le terminal, parce que c'est ce qu'on retrouve sur un serveur.

💡 Astuce diagnostic

Quand un conteneur « ne marche pas », premier réflexe : lire ses **logs** (dans Docker Desktop, ou `docker compose logs -f web`). 90 % des erreurs — mauvais port, extension manquante, faute d'indentation YAML — s'y lisent en clair.

04 Le Dockerfile

Une image toute prête comme `php:8.2-apache` suffit souvent. Mais dès qu'on veut **ajouter** quelque chose à l'image — une extension PHP, un réglage — on écrit un **Dockerfile** : une recette qui part d'une image existante et la personnalise.

Dockerfile

Un fichier texte (nommé exactement `Dockerfile`, sans extension) qui décrit, instruction par instruction, comment **construire** une image sur mesure. Chaque ligne produit une « couche » réutilisable.

Cas typique dans MyEvents : à partir du CM4, PHP doit dialoguer avec MySQL. Or l'image PHP de base n'embarque pas les extensions `pdo_mysql` / `mysqli`. On les ajoute :

Dockerfile

```
FROM php:8.2-apache
# ❶ On part de l'image officielle PHP + Apache

RUN docker-php-ext-install pdo_mysql mysqli
# ❷ On installe les extensions pour parler à MySQL depuis le code PHP
```

Les instructions que vous croiserez

Instruction	Rôle
FROM	L'image de départ. Toujours la première ligne.
RUN	Exécute une commande <i>pendant la construction</i> (installer un paquet, une extension...).
COPY	Copie des fichiers de votre projet <i>dans</i> l'image.
ENV	Définit une variable d'environnement.
WORKDIR	Choisit le dossier de travail par défaut dans le conteneur.
EXPOSE	Documente le port sur lequel l'appli écoute (informatif, ne publie rien — voir §8).
CMD	La commande lancée au démarrage du conteneur.

Dockerfile vs docker-compose.yml

Le **Dockerfile** dit *comment fabriquer une image*. Le **docker-compose.yml** dit *quels conteneurs lancer et comment les brancher ensemble*. On construit une fois, on lance autant qu'on veut.

05 Partir direct du YAML : `image` ou `build` ?

Un service a besoin d'une image pour démarrer. Il y a exactement **deux façons** de la lui donner, et on choisit l'une *ou* l'autre par service.

Façon 1 — `image:` : partir direct du YAML

On indique une image toute prête, et Compose la télécharge depuis Docker Hub. **Aucune construction**, aucun Dockerfile : le `.yaml` se suffit à lui-même.

`docker-compose.yml` (extrait)

```
web:
  image: php:8.2-apache  # on prend l'image telle quelle, direct
```

💡 C'est ce que fait notre compose élèves

Nos trois services (`web` , `db` , `phpmyadmin`) utilisent tous `image:` . Un `docker compose up -d` télécharge les images et démarre tout — **zéro construction**. Le plus simple pour démarrer, et amplement suffisant tant qu'on n'a rien à ajouter à l'image.

Façon 2 — `build:` : construire avec le Dockerfile

Quand l'image toute prête ne suffit plus (il faut une extension, un réglage), on remplace `image:` par `build:` , qui pointe vers le dossier contenant le **Dockerfile** du §4.

`docker-compose.yml` (extrait)

```
web:
  build: .  # construit l'image depuis le Dockerfile du dossier
```

🎯 Le bon réflexe

On commence **toujours** par `image:` (direct du YAML). On passe à `build:` + Dockerfile **uniquement** le jour où il faut modifier l'image — chez nous, quand PHP devra parler à MySQL et qu'il faudra `pdo_mysql` .

	<code>image:</code>	<code>build:</code> + Dockerfile
D'où vient l'image	Téléchargée toute prête (Docker Hub)	Construite chez vous
Étape de construction	Aucune	<code>docker compose build</code> (ou <code>up --build</code>)
Personnalisable	Non, on prend tel quel	Oui (extensions, réglages, fichiers copiés)
Quand l'utiliser	Par défaut, tant que ça suffit	Dès qu'il faut ajouter quelque chose

⚠ Après une modif du Dockerfile

Un simple `up` réutilise l'ancienne image construite. Pour prendre en compte un changement du Dockerfile, forcez la reconstruction : `docker compose up -d --build`.

06 Notre `docker-compose.yml`

MyEvents a besoin de **trois** conteneurs : le web (PHP + Apache), la base (MySQL) et phpMyAdmin (l'interface graphique de la base). Compose décrit tout ça dans un seul fichier, et un seul `docker compose up` allume l'ensemble.

📖 Docker Compose

Un outil qui lit un fichier `docker-compose.yml` et orchestre plusieurs conteneurs d'un coup : il les crée, les met sur un **réseau commun**, gère ports, volumes et ordre de démarrage.

Voici notre fichier complet. On le décortique bloc par bloc juste après.

```
docker-compose.yml
```

```

services:
  # Serveur web Apache + PHP 8.2 (l'"AP" de WAMP)
  web:
    image: php:8.2-apache
    container_name: myevents-web
    ports:
      - "8080:80"
    volumes:
      # Le dossier ./myevents devient la racine web
      - ./myevents:/var/www/html/myevents
      # Les scripts CLI du TP1 sont aussi montés
      - ./TP1-cli:/var/www/TP1-cli
    working_dir: /var/www/html

  # Base de données MySQL (le "M" de WAMP) – utile à partir du CM4
  db:
    image: mysql:8.0
    container_name: myevents-db
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: myevents
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql

  # phpMyAdmin (l'interface fournie par WampServer)
  phpmyadmin:
    image: phpmyadmin:latest
    container_name: myevents-pma
    ports:
      - "8081:80"
    environment:
      PMA_HOST: db
      PMA_USER: root
      PMA_PASSWORD: root
    depends_on:
      - db

volumes:
  db_data:

```

⚠ Le YAML est sensible à l'indentation

En YAML, la hiérarchie se lit **uniquement** aux espaces (jamais de tabulation !). Deux espaces = un niveau d'imbrication. Un espace en trop ou en moins, et Compose refuse le fichier. C'est l'erreur la plus fréquente.

07 Le YAML décortiqué

On reprend chaque bloc du fichier.

services: — la racine

Tout part de là. Chaque entrée en dessous (`web` , `db` , `phpmyadmin`) est un **service**, c'est-à-dire un conteneur à lancer. Le nom choisi ici sert dans les commandes *et* comme adresse réseau (voir §8).

image:

L'image téléchargée pour ce service : `php:8.2-apache` , `mysql:8.0` , `phpmyadmin:latest` . Le `:` sépare le nom de la **version (le tag)**. On fixe les versions (`8.2` , `8.0`) pour que toute la classe ait le même environnement.

container_name:

Le nom « en clair » du conteneur (`myevents-web`). Sans lui, Docker en génère un long et moche. Utile pour le Docker « pur » : `docker exec -it myevents-web bash` .

📌 Nom de service ≠ nom de conteneur

`web` est le **nom de service** (pour `docker compose ...` et le réseau). `myevents-web` est le **nom de conteneur** (pour `docker ...` tout court). Les deux existent en parallèle.

ports: — le fameux bind

Le point qui bloque tout le monde, donc on lui consacre la §8 entière. En résumé : `"8080:80"` veut dire « le port **8080 de ma machine** est relié au port **80 du conteneur** ». On lit toujours **HÔTE : CONTENEUR**.

🎯 Le sens de lecture

"8080:80" → **gauche** = **chez vous** (le numéro tapé dans le navigateur), **droite** = **dans le conteneur** (le numéro sur lequel le logiciel écoute). Jamais l'inverse.

volumes: (deux usages différents !)

Le mot « volume » recouvre deux choses selon la forme :

- **Bind mount** — `./myevents:/var/www/html/myevents` : un dossier *de votre machine* rendu visible dans le conteneur. Vous éditez le code dans VS Code, Apache le voit **instantanément**. Idéal pour le code. (On monte aussi `./TP1-cli` pour les scripts en ligne de commande.)
- **Volume nommé** — `db_data:/var/lib/mysql` : un espace géré par Docker, déclaré tout en bas dans `volumes:`. Idéal pour les **données** : la base survit à un `down`.

💡 Pourquoi c'est vital

Un conteneur est **jetable** : le supprimer efface tout ce qu'il contient. Sans `db_data`, votre base disparaîtrait à chaque `down`. Votre code, lui, est déjà à l'abri : c'est un bind mount depuis votre dossier de projet.

working_dir:

Le dossier où l'on « atterrit » quand on entre dans le conteneur ou qu'on y lance une commande. Ici `/var/www/html` : c'est de là que part `docker compose exec web ...`.

environment:

Des variables passées au conteneur au démarrage. MySQL les lit pour se configurer tout seul : il crée la base `myevents` et fixe le mot de passe root. phpMyAdmin les lit aussi (`PMA_HOST`, `PMA_USER`, `PMA_PASSWORD`) pour savoir à quelle base se connecter, et avec quel compte.

depends_on:

`phpmyadmin` dépend de `db` : Compose démarre `db` **avant** phpMyAdmin. Cela garantit l'*ordre de démarrage*, pas que MySQL soit déjà « prêt à répondre » (pour ça on ajouterait un `healthcheck`).

La section volumes: tout en bas

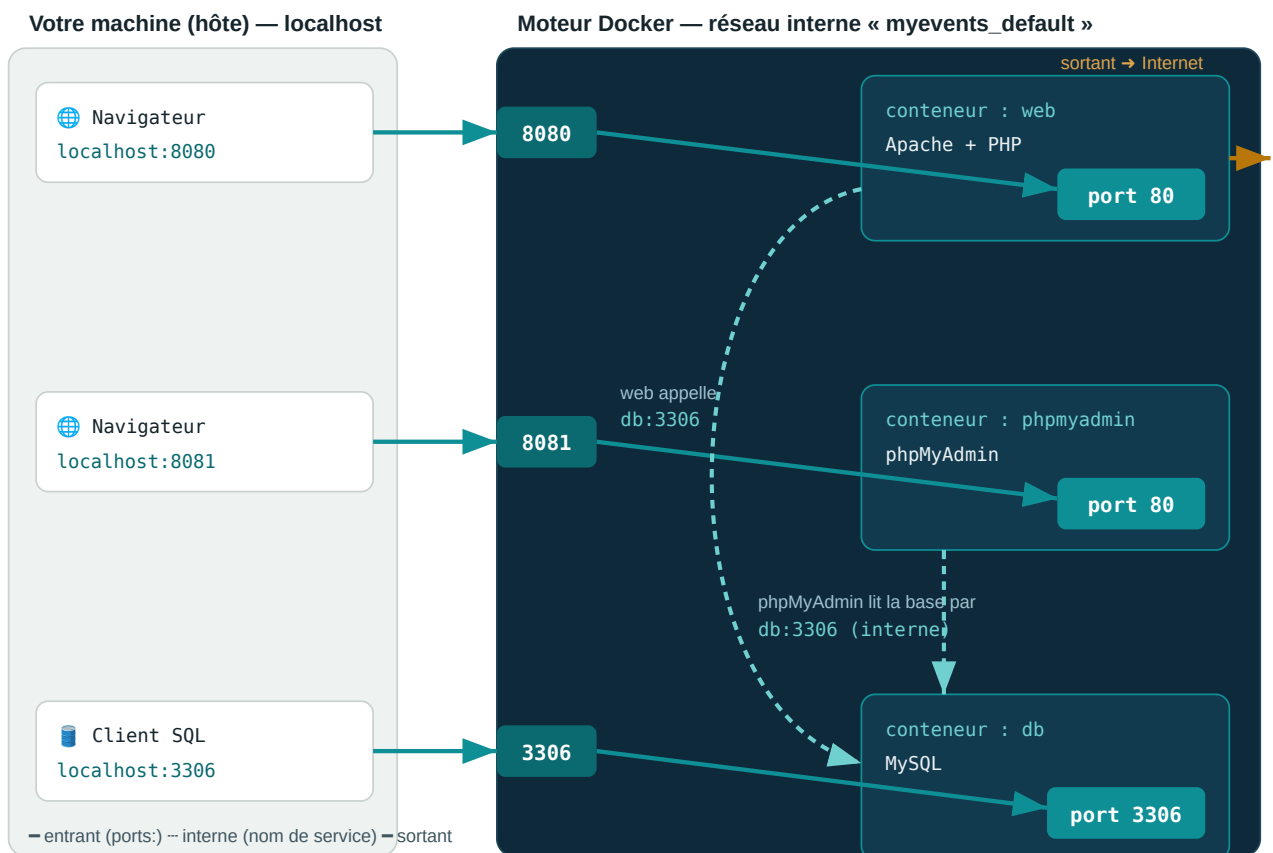
Elle **déclare** le volume nommé utilisé plus haut. `db_data:` sans rien derrière suffit : Docker le crée et gère son emplacement réel sur le disque.

08 Les ports, pour de bon

Un port, c'est un **numéro de porte** sur une machine. Un ordinateur a une seule adresse mais des milliers de portes numérotées ; chaque logiciel réseau écoute derrière une porte précise. Apache écoute derrière la 80, MySQL derrière la 3306, phpMyAdmin (qui est aussi un site web) derrière la 80 lui aussi.

Les deux côtés d'un port : conteneur et machine

Le point clé de Docker : **un conteneur a ses propres portes, séparées de celles de votre machine**. Dans le conteneur `web`, Apache écoute bien sur la porte 80 — mais cette porte est *dans le conteneur*, votre navigateur ne peut pas l'atteindre directement. Il faut **relier** une porte de votre machine à cette porte interne. C'est le rôle de `ports:`.



Trois portes publiées vers votre machine (8080, 8081, 3306). À l'intérieur du réseau Docker, `web` et `phpmyadmin` joignent `db` directement par son nom, sans passer par votre machine.

Entrant, sortant : de quel côté vient la connexion ?

« Entrant » ou « sortant » se juge **du point de vue du conteneur**, selon qui *initie* la connexion.

Sens	Qui appelle qui	Faut-il un <code>ports:</code> ?	Exemple MyEvents
Entrant (ingress)	Quelqu'un de l'extérieur se connecte <i>vers</i> le conteneur.	Oui. Publier le port ouvre la porte depuis votre machine.	Navigateur → Apache (8080 → 80).
Sortant (egress)	Le conteneur se connecte <i>vers</i> l'extérieur.	Non. Un conteneur appelle Internet librement.	<code>composer install</code> qui télécharge des paquets.
Interne (service à service)	Un conteneur du projet en appelle un autre.	Non. Le réseau Compose les relie déjà, par nom de service.	phpMyAdmin → <code>db</code> sur <code>db:3306</code> .

⚠ Les deux pièges qui coûtent le plus d'heures

- 1. Depuis le code PHP, l'hôte de la base n'est PAS `localhost`, c'est `db`.** À l'intérieur du conteneur `web`, « localhost » désigne le conteneur `web` lui-même, pas la base. On se connecte donc à `db:3306` (nom du service, port *interne*). C'est aussi pour ça que phpMyAdmin a `PMA_HOST: db` et non `localhost`.
- 2. Le port publié ne sert qu'à VOUS, depuis votre machine.** Le `3306` publié sert à brancher un outil externe (DBeaver, ligne de commande) sur la base depuis votre poste. Les conteneurs `web` et `phpmyadmin`, eux, passent par le réseau interne et se moquent de ce port publié.

💡 Conflit de port « already allocated »

Ici on publie `3306:3306`. Si un **MySQL (ou WampServer) est déjà installé et lancé** sur votre poste, il occupe déjà la porte 3306 de la machine, et Docker refusera : « port is already allocated ». Deux solutions : arrêter l'autre service, ou **décaler la gauche** — `"3307:3306"`. La droite (le port du conteneur) ne bouge jamais, elle est imposée par MySQL.

Résumé du vocabulaire des ports

Terme	Où ?	Modifiable ?	Dans MyEvents
Port du conteneur	À l'intérieur, imposé par le logiciel.	Non (fixé par Apache/MySQL).	80 , 3306
Port de l'hôte (publié)	Sur votre machine, ce que vous tapez.	Oui, vous le choisissez.	8080 , 8081 , 3306
Mapping	Le lien entre les deux, dans <code>ports:</code> .	Se lit <code>HÔTE:CONTENEUR</code> .	"8080:80"
Adresse interne	Nom de service sur le réseau Docker.	= nom du service.	db:3306

09 phpMyAdmin dans le navigateur

phpMyAdmin est l'interface graphique de la base — celle que WampServer vous donnait. Ici, c'est simplement **un conteneur de plus**, déjà branché sur `db`. Rien à installer.

Comment on l'ouvre

🎯 En trois gestes

1. Lancer le projet : `docker compose up -d`.
 2. Ouvrir le navigateur sur `http://localhost:8081`.
 3. Se connecter avec **utilisateur** `root` / **mot de passe** `root`.
- Vous voyez la base `myevents` dans la colonne de gauche.

Pourquoi 8081, et pourquoi ça marche

phpMyAdmin est lui-même un petit site web : dans son conteneur, il écoute sur le port **80**. Comme le conteneur `web` occupe déjà le 8080 côté machine, on publie phpMyAdmin sur une **autre porte libre de la machine, le 8081** :

```
phpmyadmin:
ports:
  - "8081:80"      # machine 8081 → conteneur 80 (le site phpMyAdmin)
```

Deux ports entrent en jeu, à ne pas confondre :

Connexion	Port	Publié ?	Rôle
Vous → phpMyAdmin	8081 → 80	Oui (ports:)	Afficher l'interface dans votre navigateur.
phpMyAdmin → MySQL	db:3306	Non (réseau interne)	phpMyAdmin lit et écrit dans la base.

📌 D'où vient la connexion à la base ?

De la ligne `PMA_HOST: db`. Elle dit à phpMyAdmin : « ta base s'appelle `db` » — le **nom du service**, joignable sur le réseau interne. C'est exactement le même principe que votre code PHP, qui se connectera lui aussi à l'hôte `db`, pas à `localhost`.

⚠ « This site can't be reached »

Si `localhost:8081` ne répond pas : le conteneur n'est pas démarré (`docker compose ps` pour vérifier), ou vous avez tapé `8080` (c'est le site MyEvents, pas phpMyAdmin). phpMyAdmin, c'est **8081**.

10 Adieu WAMP / XAMPP

Regardez bien notre compose : Apache + PHP (`web`), MySQL (`db`), phpMyAdmin (`phpmyadmin`). C'est **exactement** ce que WampServer ou XAMPP installaient sur votre poste — le « **W/X-A-M-P** ». Sauf qu'ici, tout est dans un fichier texte de 30 lignes.

Ce que WAMP/XAMPP faisait → son équivalent Docker

WAMP / XAMPP	Chez nous
Apache installé sur Windows	Conteneur <code>web</code> (<code>php:8.2-apache</code>)
PHP installé et configuré	Inclus dans le conteneur <code>web</code>
MySQL installé comme service Windows	Conteneur <code>db</code> (<code>mysql:8.0</code>)
phpMyAdmin livré avec WampServer	Conteneur <code>phpmyadmin</code>
Le dossier <code>www/</code>	Le bind mount <code>./myevents</code>

Pourquoi c'est mieux

- **Même environnement pour toute la classe.** Tout le monde a PHP 8.2 et MySQL 8.0, aux mêmes versions. Fini le « chez moi PHP est en 7.4 et ça plante ».
- **Rien n'est installé « en dur » sur la machine.** Pas de service Windows qui tourne en permanence, pas de réglages Apache éparpillés. Un `docker compose down` et il ne reste rien — votre poste reste propre.
- **Un projet = un dossier.** L'environnement voyage avec le code (le `.yaml` est dans le dépôt). On clone, on `up`, ça marche. Pas de doc d'installation de trois pages.
- **Plusieurs projets côte à côte.** Un projet en PHP 8.2, un autre en PHP 7.4, chacun sa base : impossible à faire proprement avec un seul WAMP, trivial avec Docker.
- **C'est la vraie vie.** En stage et en entreprise, c'est Docker que vous retrouverez, pas WampServer.

⚠ Si WAMP est encore installé sur votre poste

Ce n'est pas grave, mais **arrêtez-le avant de lancer Docker**. Un WampServer allumé occupe les ports 80 et 3306 de votre machine, et Docker ne pourra pas publier `3306:3306` (« port is already allocated »). Deux services qui veulent la même porte : quelqu'un doit céder.

💡 En clair

Vous pouvez désinstaller WAMP/XAMPP. Tout ce dont MyEvents a besoin vit désormais dans le `docker-compose.yaml` — et se lance en une commande.

11 Mini-TP : TP0 + TP1 avec Docker

On applique tout ce qui précède à un vrai TP. L'objectif : réaliser **TP0** (mettre en place l'environnement) et **TP1** (les scripts PHP en ligne de commande *et* l'application web) **entièrement avec Docker** — sans installer PHP ni Apache sur votre poste.

🎯 Ce que vous allez faire

Installer Docker → poser l'arborescence du projet → comprendre le bind mount → démarrer le serveur web + la base → exécuter les 6 scripts CLI de la Partie A *dans* le conteneur → afficher l'appli web de la Partie B dans le navigateur.

Étape 0 — Prérequis (le « TP0 » version Docker)

Le TP0 papier vous faisait installer WAMP/PHP/Apache. Ici, tout ça est déjà dans le `docker-compose.yml` : le seul logiciel à installer sur le poste, c'est **Docker** (voir §2). Vérifiez qu'il répond :

```
docker --version
docker compose version
```

⚠ Mac / Windows

Docker Desktop doit être lancé (icône 🐳 verte) avant toute commande. Sinon : « Cannot connect to the Docker daemon ».

Étape 1 — Poser l'arborescence du projet

Le `docker-compose.yml` doit se trouver **à la racine**, à côté des dossiers `myevents/` (le web) et `TP1-cli/` (les scripts). C'est indispensable : les bind mounts sont écrits en **chemins relatifs** (`./myevents`, `./TP1-cli`) à l'endroit d'où vous lancez Compose.

```
mkdir myevents-docker && cd myevents-docker
mkdir -p myevents/data myevents/inc TP1-cli
# puis placez le docker-compose.yml (fourni) à la racine du dossier
```

Vous devez obtenir :

```
myevents-docker/
├─ docker-compose.yml      # le fichier fourni
├─ myevents/               # Partie B – l'appli web
│   ├─ index.php
│   ├─ event.php
│   └─ data/events.php
├─ inc/header.inc.php , footer.inc.php
└─ TP1-cli/                # Partie A – les scripts en ligne de commande
    └─ a1_hello.php ... a6_stats.php
```

Étape 2 — Comprendre le bind mount

Deux lignes du compose relient vos dossiers du poste à l'intérieur du conteneur `web` :

Sur votre poste	Dans le conteneur <code>web</code>	Pour quoi
<code>./myevents</code>	<code>/var/www/html/myevents</code>	servi par Apache → <code>localhost:8080/myevents/</code>
<code>./TP1-cli</code>	<code>/var/www/TP1-cli</code>	exécuté en ligne de commande

💡 L'intérêt concret

Vous éditez vos fichiers **normalement dans VS Code**, sur votre poste. Le conteneur voit les changements **instantanément** : pas besoin de recopier quoi que ce soit ni de redémarrer. Enregistrez, rechargez le navigateur.

Étape 3 — Démarrer le serveur (web + base)

```
docker compose up -d      # 1re fois : télécharge les images, patientez
docker compose ps         # web, db, phpmyadmin doivent être "running"
```

À partir de là, Apache + PHP tournent. Le TP0 papier demandait un « `php -v` qui répond 8.x » : ici, PHP 8.2 est *dans* le conteneur, garanti identique pour toute la classe.

Étape 4 — Vérifier que le web répond (fin du TP0)

Reprenons le test `phpinfo` du TP0. Créez `myevents/phpinfo.php` :

```
<?php phpinfo();
```

Ouvrez dans le navigateur : `http://localhost:8080/myevents/phpinfo.php`. Vous devez voir la page de configuration de PHP.

⚠️ Deux réflexes

1. L'adresse n'est plus `localhost/myevents/` (port 80, WAMP) mais `localhost:8080/myevents/` — le port publié par le compose.
2. Supprimez `phpinfo.php` dès la vérification faite : il expose des informations serveur sensibles. C'est le premier réflexe de sécurité du module.

Étape 5 — Partie A : exécuter les scripts CLI *via Docker*

Les scripts de `TP1-cli/` ne se lancent plus avec `php a1_hello.php` sur votre poste, mais à **l'intérieur du conteneur**. Le plus simple : entrer dedans une fois, puis enchaîner.

```
docker compose exec web bash      # on entre dans le conteneur web
cd /var/www/TP1-cli
php a1_hello.php
php a2_fiche.php
php a6_stats.php
exit                               # ressortir ; Apache continue de tourner
```

⚠ Les scripts interactifs A3 et A5

`a3_places.php` et `a5_liste_titres.php` lisent le clavier avec `fgets(STDIN)`. Pour qu'ils reçoivent bien votre saisie, lancez-les avec l'option `-it` (interactif + terminal) :

```
docker exec -it myevents-web php /var/www/TP1-cli/a3_places.php
```

📌 Pourquoi `-it` ici ?

`-i` garde l'entrée clavier ouverte, `-t` alloue un vrai terminal. Sans eux, `fgets(STDIN)` ne « voit » aucune frappe et le script tourne dans le vide. Notez qu'on utilise `myevents-web` : c'est le **nom du conteneur** (défini par `container_name`), pas le nom du service.

Étape 6 — Partie B : l'application web

1. Éditez `myevents/index.php`, `event.php`, `data/events.php` et les `inc/` dans VS Code, sur votre poste.
2. Rechargez `http://localhost:8080/myevents/` dans le navigateur. Grâce au bind mount, chaque enregistrement est pris en compte **sans rien redémarrer**.
3. Le style W3.CSS est chargé par le navigateur depuis Internet (le `<link>` du header) : rien de spécial à faire côté Docker.

💡 Une erreur PHP à l'écran ?

Suivez les logs Apache/PHP en direct dans un autre terminal : `docker compose logs -f web`. Les *Parse error*, *Undefined array key* & co. y apparaissent en clair.

Étape 7 — Arrêter en fin de séance

```
docker compose stop # éteint tout, garde vos données et votre code
```

Ce qui change par rapport au sujet papier

Le TP0/TP1 papier disait...	...avec Docker, ça devient
Installer WAMP / Homebrew / <code>apt install php</code>	Installer Docker, puis <code>docker compose up -d</code>
<code>php a1_hello.php</code> (sur le poste)	<code>docker compose exec web php /var/www/TP1-cli/a1_hello.php</code>
Script interactif : <code>php a3_places.php</code>	<code>docker exec -it myevents-web php /var/www/TP1-cli/a3_places.php</code>
Web : <code>http://localhost/myevents/</code>	Web : <code>http://localhost:8080/myevents/</code>
Dossier <code>www</code> de WAMP	Bind mount <code>./myevents</code>

Checklist de fin (version Docker)

✓ Vous avez terminé quand...

- ☐ `docker --version` répond, et Docker Desktop est lancé (Mac/Win).
- ☐ `docker compose up -d` puis `docker compose ps` : `web`, `db`, `phpmyadmin` sont *running*.
- ☐ `http://localhost:8080/myevents/` est accessible ; `phpinfo.php` a été testé **puis supprimé**.
- ☐ Les 6 scripts de `TP1-cli/` s'exécutent via Docker, sans erreur ni avertissement (dont A3 et A5 en `-it`).
- ☐ `index.php` affiche le message de bienvenue, la date du jour, la liste des événements via `foreach` et les badges de disponibilité corrects.

12 Récap des commandes

À lancer depuis le dossier qui contient le `docker-compose.yml`.

Cycle de vie

Commande	Effet	Analogie
<code>docker compose up -d</code>	Démarre tous les services en arrière-plan (<code>-d</code> = detached). Télécharge/crée les conteneurs si besoin.	Allumer
<code>docker compose ps</code>	Liste les conteneurs du projet et leur statut (<code>running</code> = OK).	Le voyant vert
<code>docker compose stop</code>	Arrête les conteneurs sans les supprimer . Les données restent.	Éteindre
<code>docker compose start</code>	Rallume des conteneurs arrêtés (plus rapide que <code>up</code>).	Rallumer
<code>docker compose restart</code>	Arrête puis redémarre (utile après un changement de config).	Redémarrer
<code>docker compose down</code>	Arrête et supprime les conteneurs (le code sur le disque reste).	Démonter
<code>docker compose down -v</code>	Idem + efface les volumes (donc la base MySQL !).	Démonter + reformater

Construire & mettre à jour

Commande	Effet
<code>docker compose pull</code>	Met à jour les images « toutes prêtes » (comme <code>mysql:8.0</code>).
<code>docker compose build</code>	(Re)construit les images qui ont un <code>build:</code> + Dockerfile.
<code>docker compose up -d --build</code>	Reconstruit puis démarre. À faire après une modif du Dockerfile.

Travailler dans un conteneur & diagnostiquer

Commande	Effet
<code>docker compose exec web bash</code>	Ouvre un terminal dans le service <code>web</code> (<code>exit</code> pour sortir).
<code>docker compose exec web php -v</code>	Lance une seule commande dans <code>web</code> et rend la main.
<code>docker compose logs -f web</code>	Suit les logs de <code>web</code> en direct (<code>-f</code> = follow, <code>Ctrl+C</code> pour quitter).
<code>docker compose ps</code>	Statut des services du projet.
<code>docker ps</code>	Liste tous les conteneurs de la machine (pas que ce projet).

🎯 Mémo express — les 6 vitales

```

docker compose up -d      # allumer
docker compose ps         # vérifier
docker compose exec web bash # entrer dedans
docker compose logs -f web # voir les erreurs
docker compose stop       # éteindre
docker compose down       # tout démonter

```

📌 Le mot de la fin

Les conteneurs sont **jetables**, votre travail ne l'est pas : le code vit dans votre dossier (bind mount) et la base dans le volume `db_data` . Vous pouvez `down` puis `up` autant de fois que vous voulez sans rien perdre — sauf si vous ajoutez le `-v` .

Cours Docker & Docker Compose — support de TP, projet **MyEvents**.

Astuce impression : `Ctrl/Cmd + P` → « Enregistrer en PDF » pour la version papier.